

Introduction to Object Oriented System Design

Object is combination of data & logic that represents some real world entity

Ex:- Car

Two Orthogonal views of the software:-

There are two ^(views) approaches used to develop software

- * Traditional view (Approach)
- * Object oriented view

Traditional Approach

- Software is considered as combination of program & data structure
- It mainly focuses on functions of system
- Moving from one phase to another phase is complex
- Increases duration of projects

* Object Oriented Approach

- Software is considered as combination of algorithm & data structures
- It focuses on object (Data & Functionality)
- Moving from one phase to another phase is easier

* It Decreases duration of projects

* Static binding:- The process of determining at ~~unit~~ ~~time~~ ~~at~~ ~~me~~ ~~whi~~ compile time, which function to invoke is called Static binding. It optimizes the calls.

* Dynamic binding:- The process of determining at runtime which function to invoke is called Dynamic binding. It occurs when polymorphic calls are issued. Not all calls function invocations require dynamic binding.

2

* Why is object orientation necessary?

Object Oriented methods enables to create sets of objects that work together to produce software that better model their problem domains than similar systems produced by traditional techniques

Reasons why object orientation works

* Higher Level of Abstraction (working will be hidden)

The top-down approach supports abstraction at the functional level

The object oriented approach supports abstraction at object level. Since objects encapsulate both data & functions they work at a higher level of abstraction. This makes designing, coding, testing of system much simpler

* Seamless transition among different phases of software development :-

The software Traditional approach to software development requires different styles of process. Moving from one phase to another phase is complex. This transition is not only slow the development process but also increases the size of project

The object oriented approach uses same language to talk about analysis, design, programming. This Seamless approach reduces level of complexity & makes clearer system development

- * Encouragement of good programming techniques
A class in an object oriented system carefully delineates b/w its interface (specifications of what the class can do) & implementation of that interface (how the class does what it ~~can~~^{does} do).

The routines & attributes within a class are held together tightly. In a properly designed system, classes will be grouped into subsystems, but remain independent $\therefore$ changing one class has no impact on other classes.

- * Promotion of reusability:- Objects are reusable because they are modeled directly out of real-world problem domain. Each object stands by itself or within a small circle of other objects. The class does not concern itself with rest of system or how it is going to be used within a particular system. This means that classes are designed generically, with reuse as a constant background goal.

Difference b/w Traditional & Object oriented approach

<u>Traditional</u>	<u>Object oriented</u>
* Software = program + data structure	* Algorithm + data structure = Software
* Focuses on functions	* Focuses on objects
* Moving from one phase to another is complex	* Moving from one phase to another is easier
* Increases duration of project	* Decreases duration of project
* Abstraction at functional level	* Abstraction at higher level
*	*

Object Oriented Programming

A language is object oriented if & only if it satisfies following requirements

- * Supports objects
- * Objects have a class
- * Class may inherit attributes from super class

Object oriented Analysis :

It is a method of analysis that examines requirements from the users

Object oriented Design

It leads to an object oriented decomposition
Uses different notations to express different models

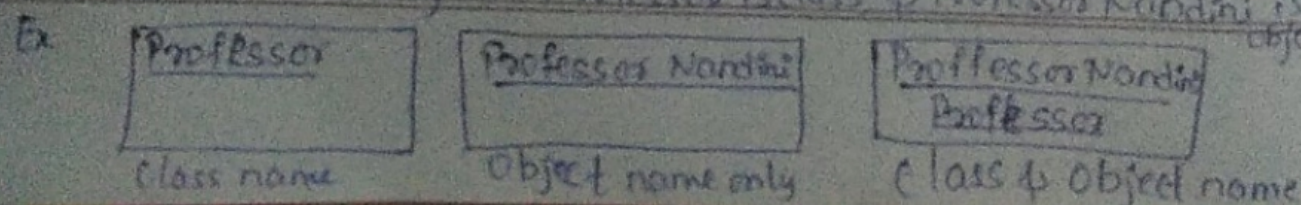
Basic Concepts of Object Orientation :-

- * Object
- * Class
- * Attribute (Property)
- * Method (Procedure)
- * Inheritance
- * Interface (Polymorphism)
- * Relationships

Object :- It is a combination of data & logic that represents some 'real world' entity, either physical, conceptual, or software

- Ex Physical entity - car
- Conceptual entity - chemical process
- Software entity - linked list

An object is represented as rectangles with underlined names. In following ex. Professor is class & Professor Nandini is object



5

* Class:- A class is a set of objects with common properties (attributes), behavior (operations), & relationships. An object is an instance of a class.

→ A class is represented using a compartmented rectangle.

→ A class is comprised of three sections

- i] The 1st section contains the class name
- ii] The 2nd section shows the structure (attributes)
- iii] The 3rd section shows the behavior (operations).

Ex:-	Classname ←	Professor
	Attributes ←	name empID
	Operations ←	{ create() save() delete() change()

Relationship between classes & objects :- Objects are grouped into class

Ex. class Professor

Professor Smith	} objects of class professor
Professor Mellon	
Professor Jones	

* Attribute :- Properties, these describe the state (data) of an object. For example properties of car are color, manufacturer, cost etc.

Ex. Class ←	Car
Attributes ←	{ cost color manufacturer

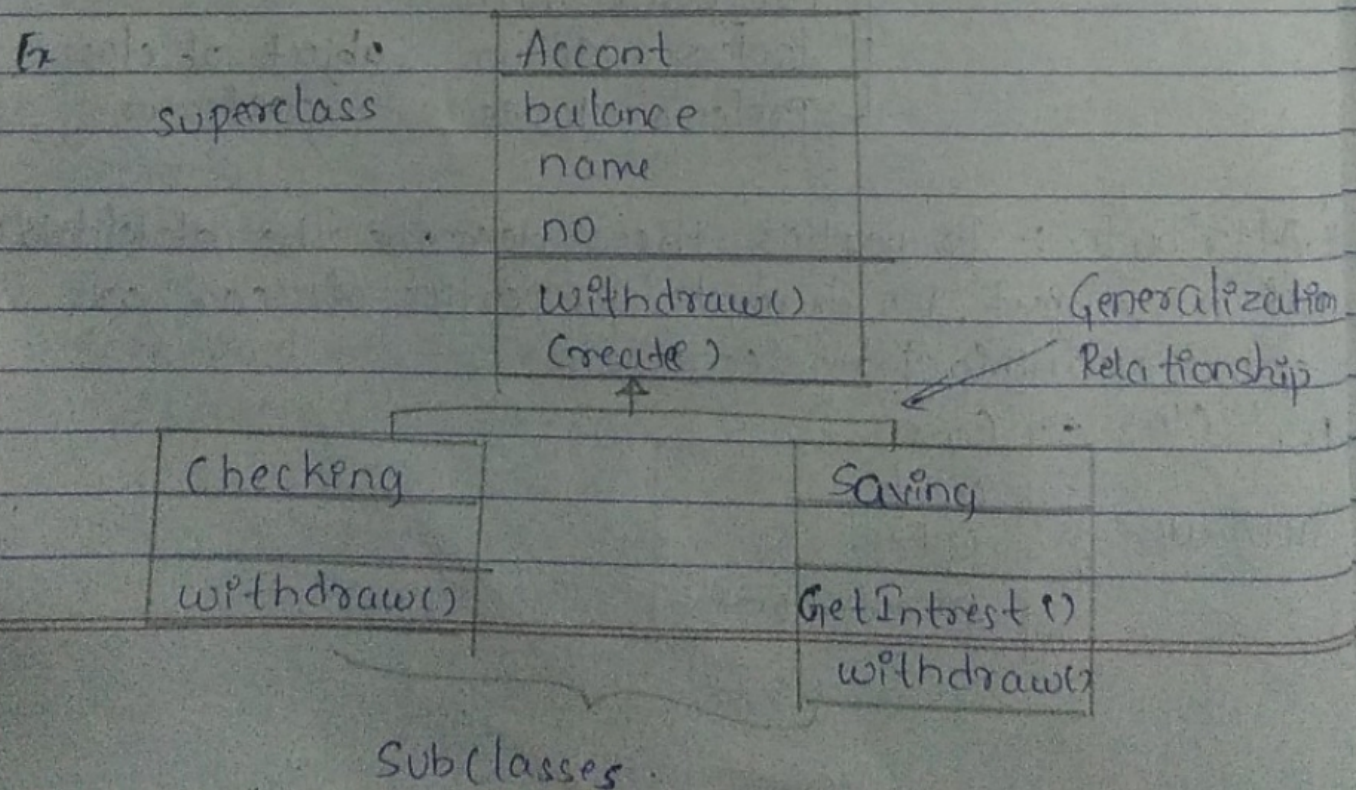
6

* Methods (procedure) :- Define its behavior (wt object is capable of doing).
 Behavior denotes the collection of methods that abstractly describes a particular behavior of an object.

Ex Class	Course offering
operation {	addStudent deleteStudent getStart time get Endtime

* Inheritance :- It allows objects to be built from other objects. It is a relationship b/w classes.
 One class is parent of other derived class.
 The parent class is known as base class or superclass.
 The main advantage is that we can build on what we already have & more important, reuse what we have.

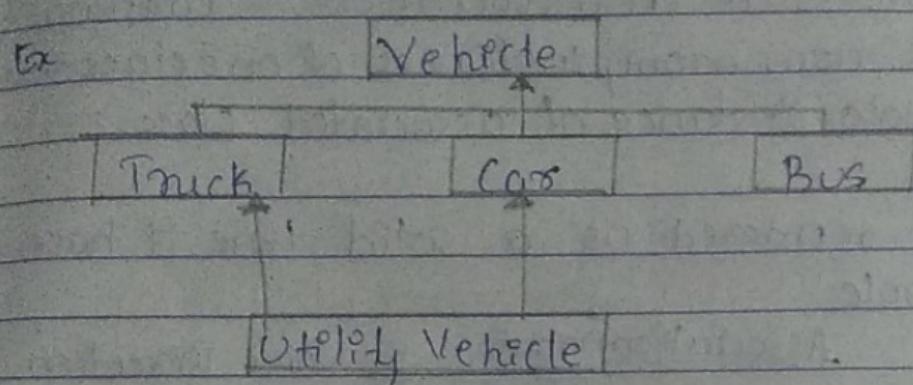
Single Inheritance :- One class inherits from another



7

Account is superclass & Checking & saving are subclasses which inherits properties of class-account. This relationship is known as Generalization relationship & denoted by whole arrow head.

Multiple Inheritance:- Some object oriented system permit a class to inherit its state (attributes) & behaviours from more than one superclass.



* Polymorphism:- Poly means many, morph means form. Polymorphism means the same operation may behave differently on different classes.

Borich defines Polymorphism as relationship of object & of many different classes by some common superclass.

Polymorphism allows us to write generic, reusable code more easily, because we can specify general instructions & delegate the implementation details to objects involved. Since no assumption is made about class of an object that receives a message.

* Relationships :-

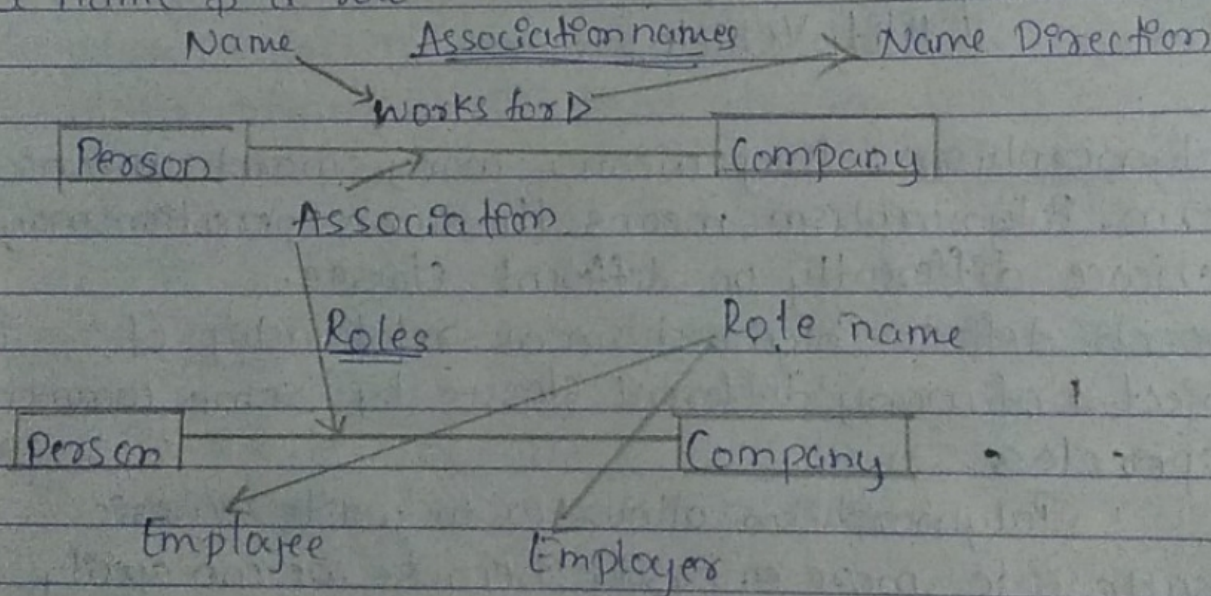
- Association
- Dependency
- Generalization Realization
- + Aggregation

Association :- It represents the relationship between objects & class. Association is bidirectional, that means they can be traversed in both directions.

The direction implied by name is forward direction the opposite direction is inverse direction.

An important issue in association is cardinality, which specifies how many instances of one class may relate to a single instance of associated class

Association is rendered as a solid line, it has a name & a role

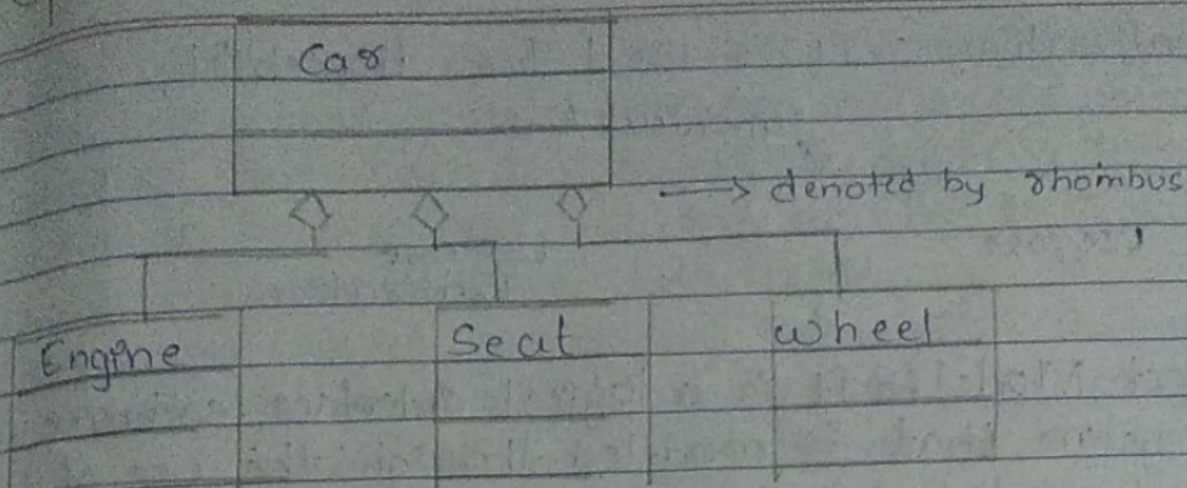


Aggregation :- A special form of association, that models a whole part relationship b/w an aggregate (the whole) & its parts.

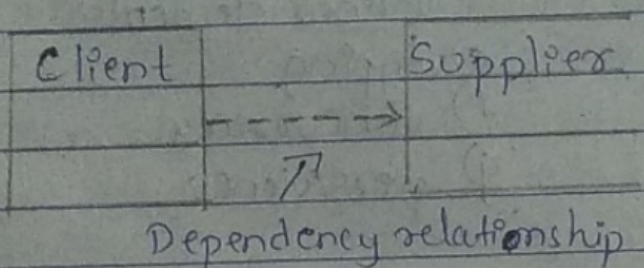
(Aggregation - A whole formed by small parts)

Ex car is object of other objects such as engine, seat & wheel objects

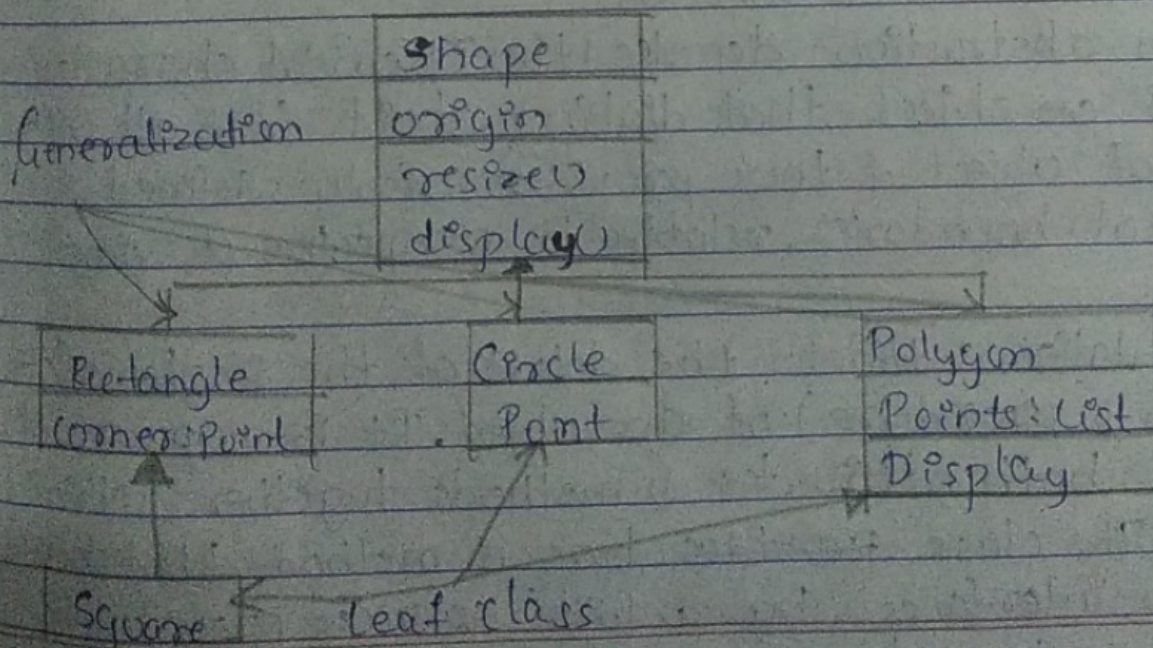
9



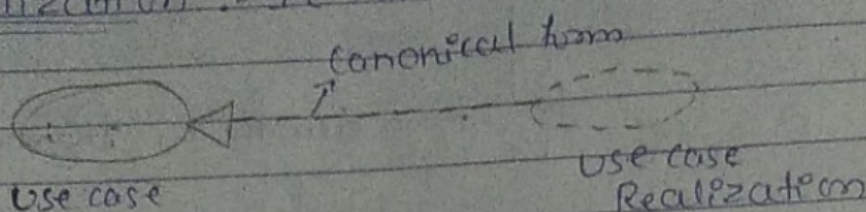
Dependency :- A relationship b/n two model elements where a change in one may cause a change in others.



Generalization :- is a kind of relationship. It is a relationship b/n a general thing (superclass) & a more specific kind of that thing (subclass).



* Realization :- It is used for traceability



Object Model :- It is a logical interface software or system that is modeled through the use of object oriented techniques. It is part of object-oriented programming lifecycle.

The main principles of object model are

Major elements

- abstraction
- encapsulation
- modularity
- hierarchy

Minor elements

- typing
- concurrency
- persistence

* Abstraction :- It means to focus on the essential features of an element or object in OOP

Booch defines abstraction as

An abstraction denotes the essential characteristics of an object that distinguish it from all other kinds of object & thus provides crisply defined conceptual boundaries relative to perspective of viewer

* Encapsulation :- Hides the details of the implementation (data & procedure) of an object. It is a process of binding both attributes & methods together within a class. The class provides has a method that provide user interfaces by which services provided by class may be used

11
Object is said to encapsulate data & program. This means that user cannot see inside of object "capsule", but can use object by calling object methods. Rather than allowing an object direct access to another object's data, a message is sent to the target object requesting information. This ensures not only that instructions are operating on proper data, but also that no object can operate directly on another object's data.

* Modularity :- Breaking up of something complex into manageable pieces.

Effective modular design :- Cohesion

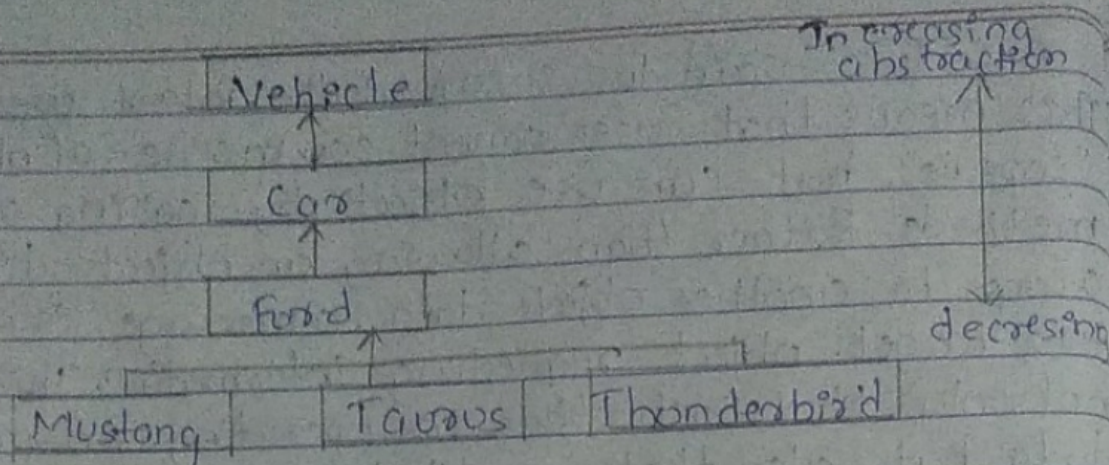
1. Cohesion 2. Coupling

Modularity is intrinsically linked with encapsulation. It can be visualized as a way of mapping encapsulated abstractions into real, physical modules having high cohesion within the modules & their inter-module interaction or coupling is low.

* Hierarchy :- Is Ranking or ordering of abstractions. At the top of the hierarchy are most general classes. At the bottom are most specific.

* A subclass inherits all of the properties & methods from a superclass.

Elements at the same level of hierarchy should be at same level of abstraction. Through hierarchy a system can be made up of interrelated subsystems & so on which can have their own subsystems & so on until smallest level of components are reached.



* Typing :- Typing is the enforcement of the ^{object} notion such that objects different types may not be interchanged, or at the most they may be interchanged only in very restricted ways.

Strong typing :- Here operation on an object is checked at time of compilation. It does prevents mixing abstractions.

Weak Typing :- Here message may be sent to any class. The operation is checked only at time of execution.

* Concurrency :- It allows different objects to act at the same time. It's the property that distinguishes an active object from one that is not active.

Concurrency in OS allows performing multiple tasks, or processes performing. When single process exist it is said that there is a single thread of control. Most of systems have multiple threads, some active, some waiting of CPU, some suspended & some terminated. System on single CPU use appropriate algorithms to give equitable CPU time to threads so as to enable concurrency.

* Persistence :- It is the property of an object through which the object continues to exist after its creator exists or the object's location moves from address phase in which it was created. Object lifespan is longer than the duration of process creating the object. ~~in an oop.~~

Software Development Life cycle

- * System development refers to all activities that produce the solution.
- * System development activities consist of analysis, modeling, design, implementation, testing & maintenance.
- * A software development methodology is a series of processes that, if followed, can lead to the development of an application.

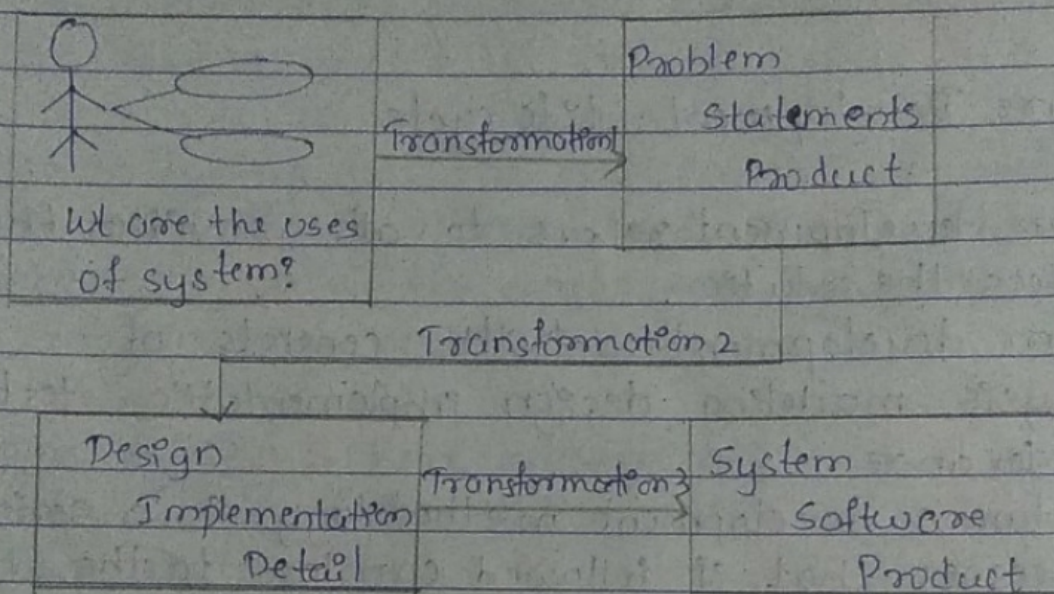
Software Development Process Transformation

Waterfall model

Software development process also can be divided into smaller, interacting subprocesses. Generally, the software development process can be viewed as a series of transformations, where the output of one transformation becomes the input of the subsequent transformation.

* Transformation 1 (analysis) :- It translates the user's needs into system requirements & responsibilities.

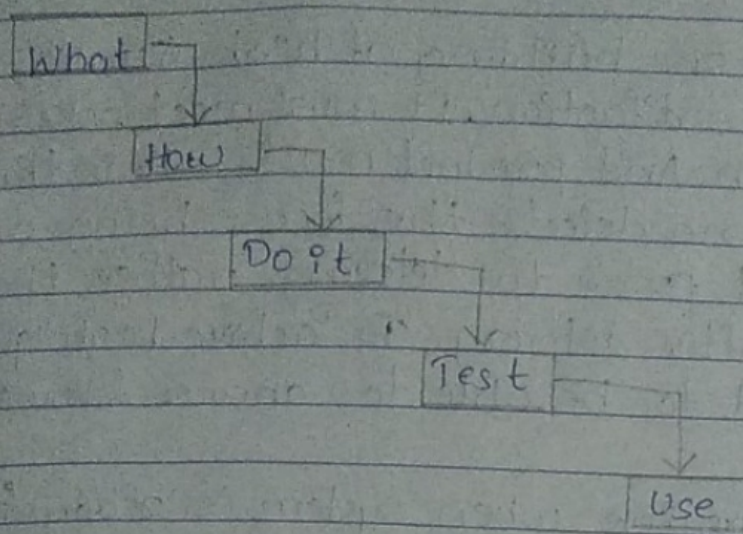
The way they use the system can provide insight into the user's requirements. For example, one use of system might be analyzing an incentive payroll system, which will tell us that this capacity must be included in the system requirements.



Transformation 2 (design) :- It begins with a problem statement & ends with detailed design that can be transformed into an operational system. This transformation includes bulk of the software development activity, including definition of how to build software, its development & its testing. It also includes design descriptions, program & testing materials.

Transformation 3 (implementation) :- It refines the detailed design into the system deployment that will satisfy the user's needs. This takes into account the equipment, procedures, people & the like. It represents embedding the software product within its operational environment.

Linear Sequential Model or Waterfall model



An example of software development process is the waterfall approach, which starts with deciding what is to be done (what is problem). Once the requirements have been determined, we next must decide how to accomplish them. This is followed by a step in which we do it, whatever it has required us to do. We then must test the result to see if we have satisfied the users requirements. Finally, we use what we have done.

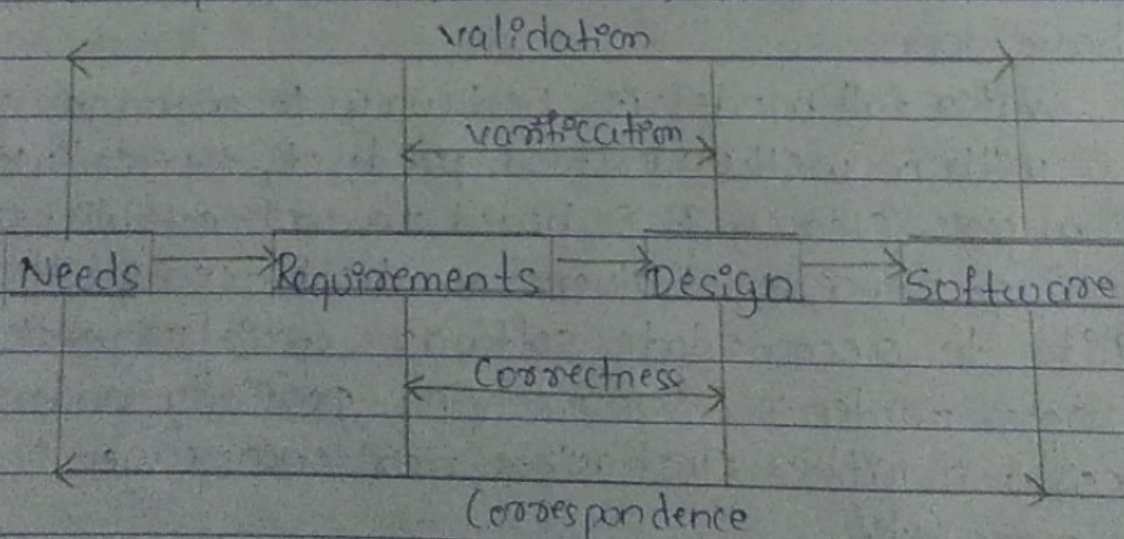
Waterfall model is best way to manage a project with a well-understood product, especially very large projects. Clearly it is based on well-established engineering principles. Failures can be traced to its inability to accommodate software's special properties & its inappropriateness for resolving partially understood issues, & it neither emphasizes nor encourages software reusability.

The Building of High-Quality Software

The main goal of building of high Quality software is user satisfaction. It must meet users needs & expectation. And product must attain this with minimal or no defects, the focus being on improving product prior to delivery rather than correcting them after delivery. To achieve high quality product we need to be able to answer following questions

- 1] How do we determine when system is ready for delivery
- 2] Is it how an operational system that satisfies users needs?
- 3] Is it correct & operating as we thought it should?
- 4] Does it pass an evaluation process

System evaluation can be done by 4 quality measures i.e. correspondence, correctness, validation & verification



* Correspondence :- It measures how well the delivered system matches the needs of operational environment, as described in the original requirements statements.

* Correctness :- It measures the consistency of the product measure requirements with respect to design specification.

* Validation :- It is task of predicting correspondence. True correspondence cannot be determined until the system is in place.

* Verification :- It is exercise of determining correctness.

Verification & Validation answers following questions

- * Verification - Am I building product right?
- * Validation - Am I building right product?

Verification & Validation are independent to each other. Verification begins as soon as ^{the} project starts, while validation begins only after a specification has been accepted.

Object Oriented System Development :- Use case driven approach [SDLC]

The object-oriented system development life cycle consists of three macro process : object oriented analysis, object-oriented design & object oriented implementation.

DOSD includes these activities

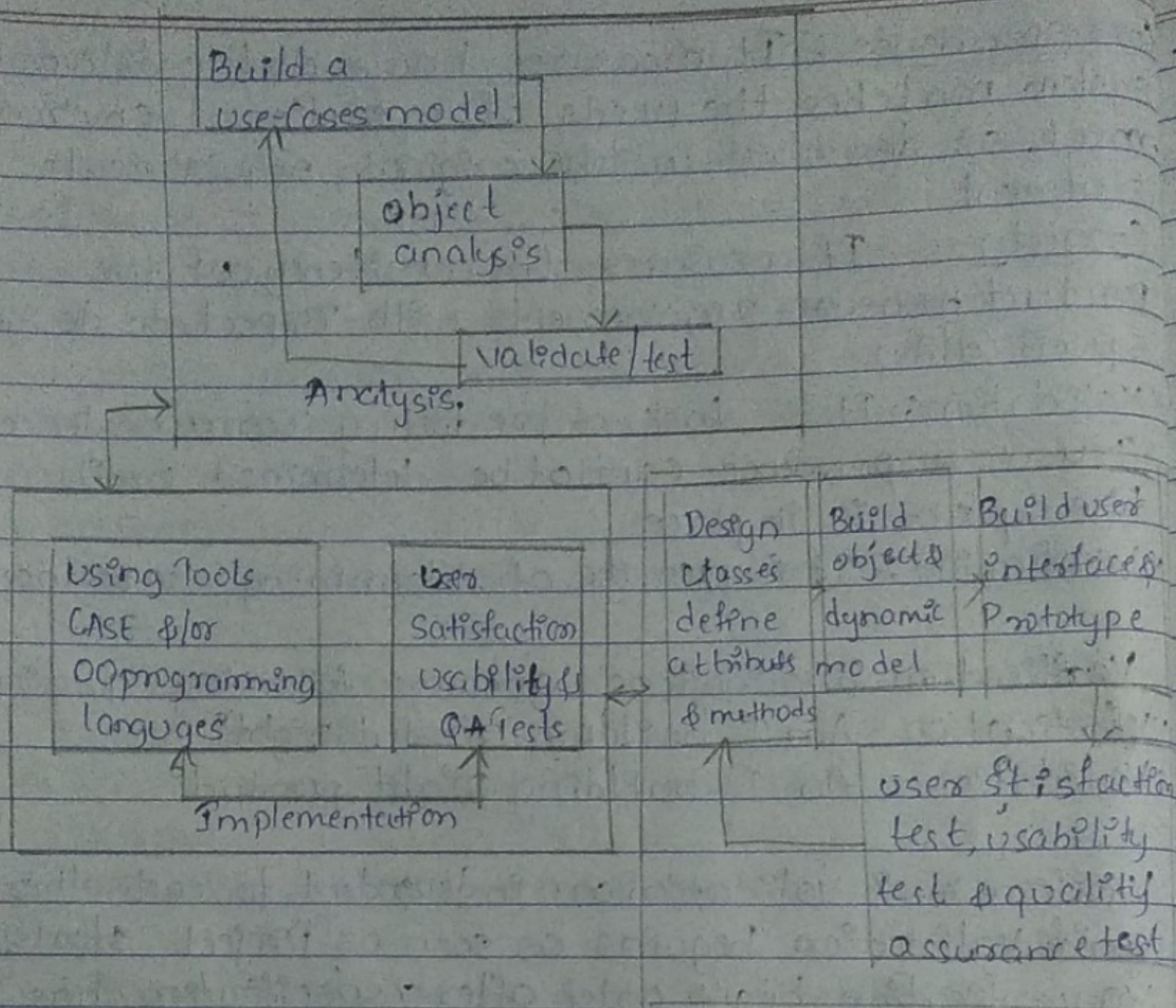
1) UOA (use case driven)

2) OOD

3) Prototyping

4) Component based development

5) Incremental testing.



Object oriented Analysis - Use case driven :-

The OOA phase of software development is concerned with determining the system requirements & identifying classes & their relationship to other classes in the problem domain.

To understand the system requirements, we need to identify the users or actors, who are the actors & how do they use the system?.

Use case is a typical interaction b/w a user & a system that captures users goals & needs. Expressing these high-level processes & interactions with customers in scenario & analyzing it is referred to as use-case modeling. The use-case model represents users view of the system or users needs.

* Object-Oriented Design :- The goal of OOD is to design the classes identified during analysis phase & user interface. During this phase, we identify & define additional objects & classes that support implementation of the requirements.

OOD & OOA are distinct disciplines, but they can be intertwined. Object-oriented development is highly incremental, in other words, you start with OOA model & create an OOD.

1st build the object model based on objects & their relationships, then iterate & refine model

- Design & refine classes
- Design & refine attributes
- Design & refine methods
- Design & refine structures
- Design & refine associations

OOD Guidelines :-

- * Reuse rather than build a new class
- * Design a large no. of simple classes rather than a small no. of complex classes
- * Design what you do methods
- * Critique what you have proposed, if possible, go back & refine the classes.

* Prototyping :- Prototyping enables to fully understand how easy or difficult it will be to implement some of the features of system.

It can also give users a chance to comment on the usability & usefulness of the design. It can further define a use-case & make use-case modeling much easier.

Prototypes have been categorized as follows

- A horizontal prototype :- It is a simulation of the interface but contains no functionality.
- A vertical prototype :- is a subset of the system features with complete functionality.
- An analyst's prototype :- is an aid for exploring problem domain. This class of prototype is used to inform users & demonstrate proof of a concept.
- A domain prototype is an aid for the incremental development of ultimate software solution.

* Implementation: Component-Based Development

CBD is an industrialized approach to the software development process. Application development moves from custom development to assembly of prebuilt, pretested reusable software components that operate with each other.

Two basic ideas underlie CBD. 1st the application development can be improved significantly, if applications can be assembled quickly from prefabricated software components. Second, an increasingly large collection of interpretable software components could be made available to developers in both general & specialist catalogs. CBD focuses on reusability.

* Incremental testing :- Software development & all of its activities including testing are an iterative process.

Therefore, it can be a costly affair if we wait to test a product only after its complete development. Here incremental testing comes into picture where in the product is tested during various stages of its development.